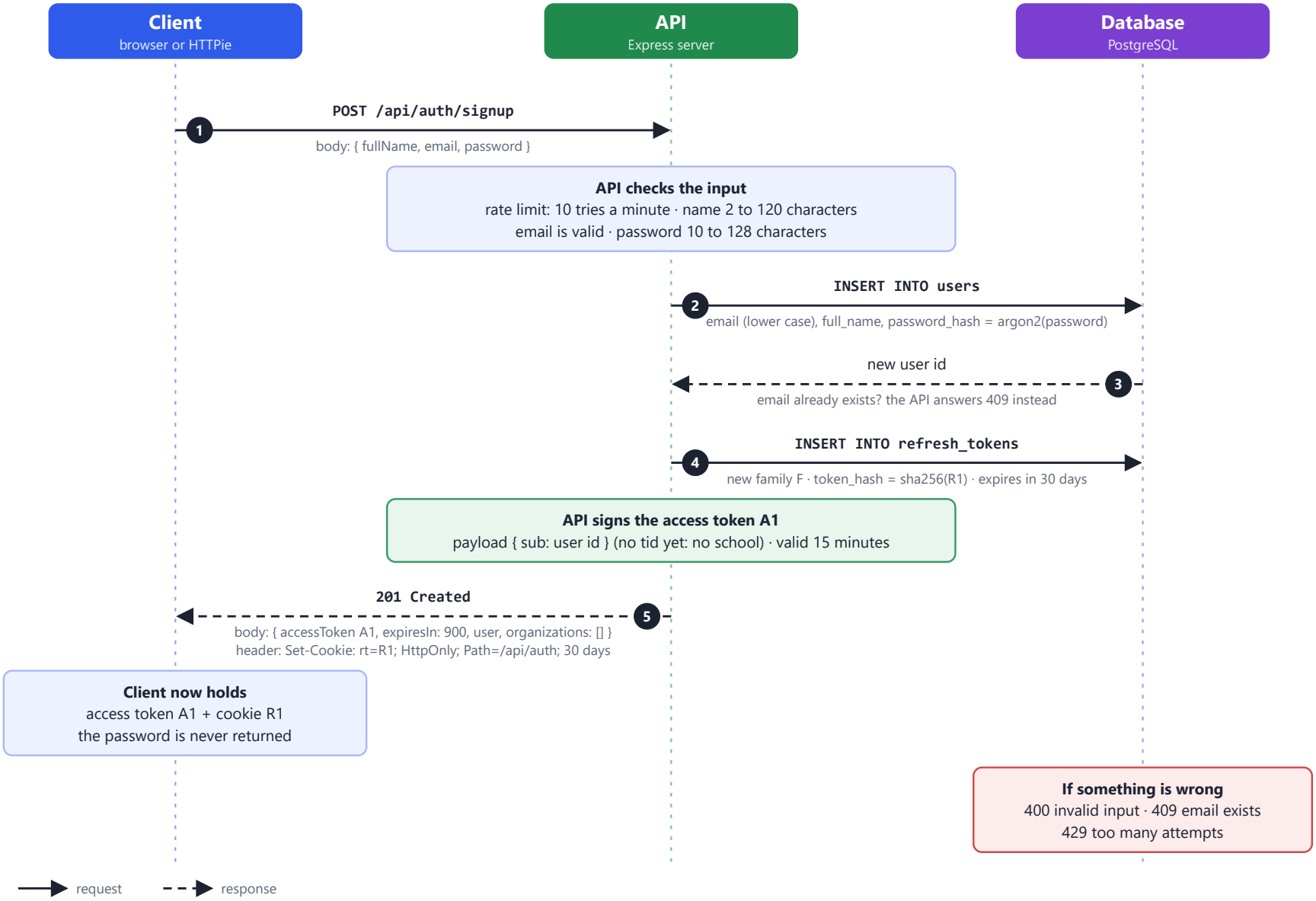
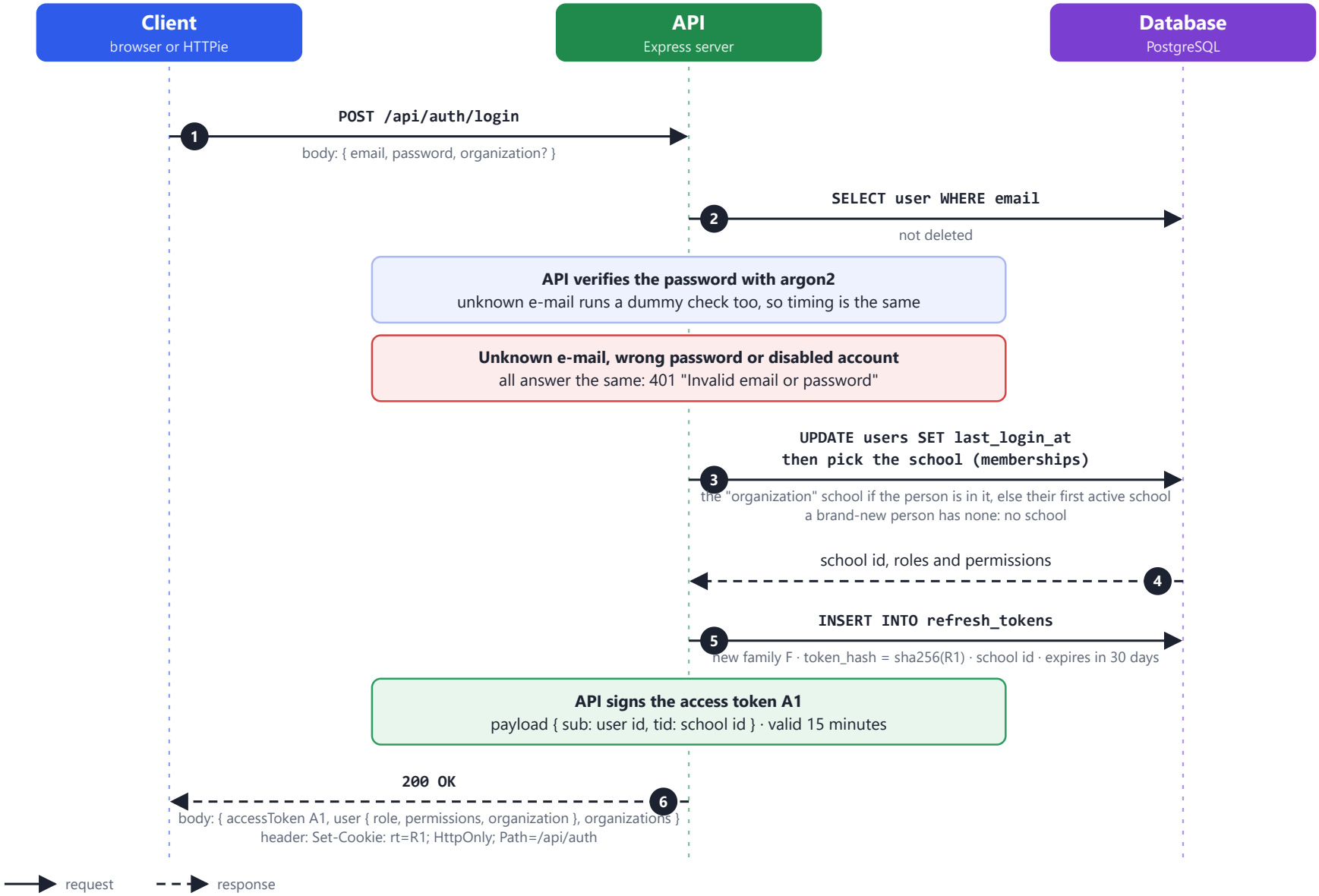
1. Sign up: creating a user

One request creates the account **and** starts a session. The new person has no school yet.



2. Sign in: starting a session

Same two tokens as sign-up, but now the access token also names the school the person is working in.



3. The two tokens

What each one is, where it travels, and where it is stored.

Access token

the room keycard

`header` . `payload` . `signature`

It is	a signed JWT. Anyone can read it, nobody can forge it.
Contains	`sub` user id · `tid` school id (absent until a school is chosen) · `iat` / `exp`
Lives	**15 minutes**
Travels	comes back in the JSON body; you send it in the header `Authorization: Bearer …` on every call
Client keeps it	in the app's memory (browser) or in the HTTPie session file
Server stores	**nothing**: it checks its own signature
Each request	verify signature and expiry, then **reload** the user, membership and permissions from the database

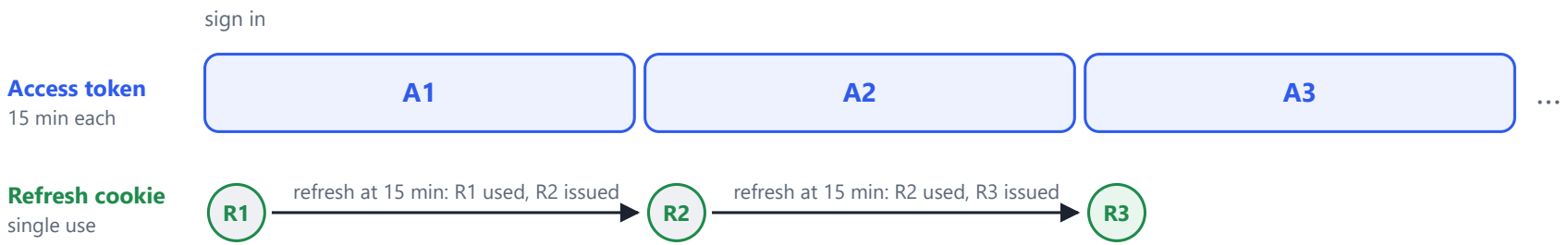
Refresh token

the pass for a new keycard

`rt = gO3kVn2Xq8… (64 random characters)`

It is	a random lookup key. It contains no information.
Lives	**30 days**, but **single use**: each refresh swaps it for a new one
Travels	`Set-Cookie: rt=…` with `HttpOnly`, `SameSite=Lax`, `Path=/api/auth` (and `Secure` on https). Sent back automatically, only to `/api/auth/*`
Client keeps it	in the browser's cookie jar (JavaScript cannot read it) or in the HTTPie session file
Server stores	table `refresh_tokens`: **sha256 hash** of the token, user, school, `family_id`, `expires_at`, `used_at`, `revoked_at`, `revoked_reason`, device
Used for	only `/auth/refresh`, `/auth/logout`, `/auth/switch-organization`

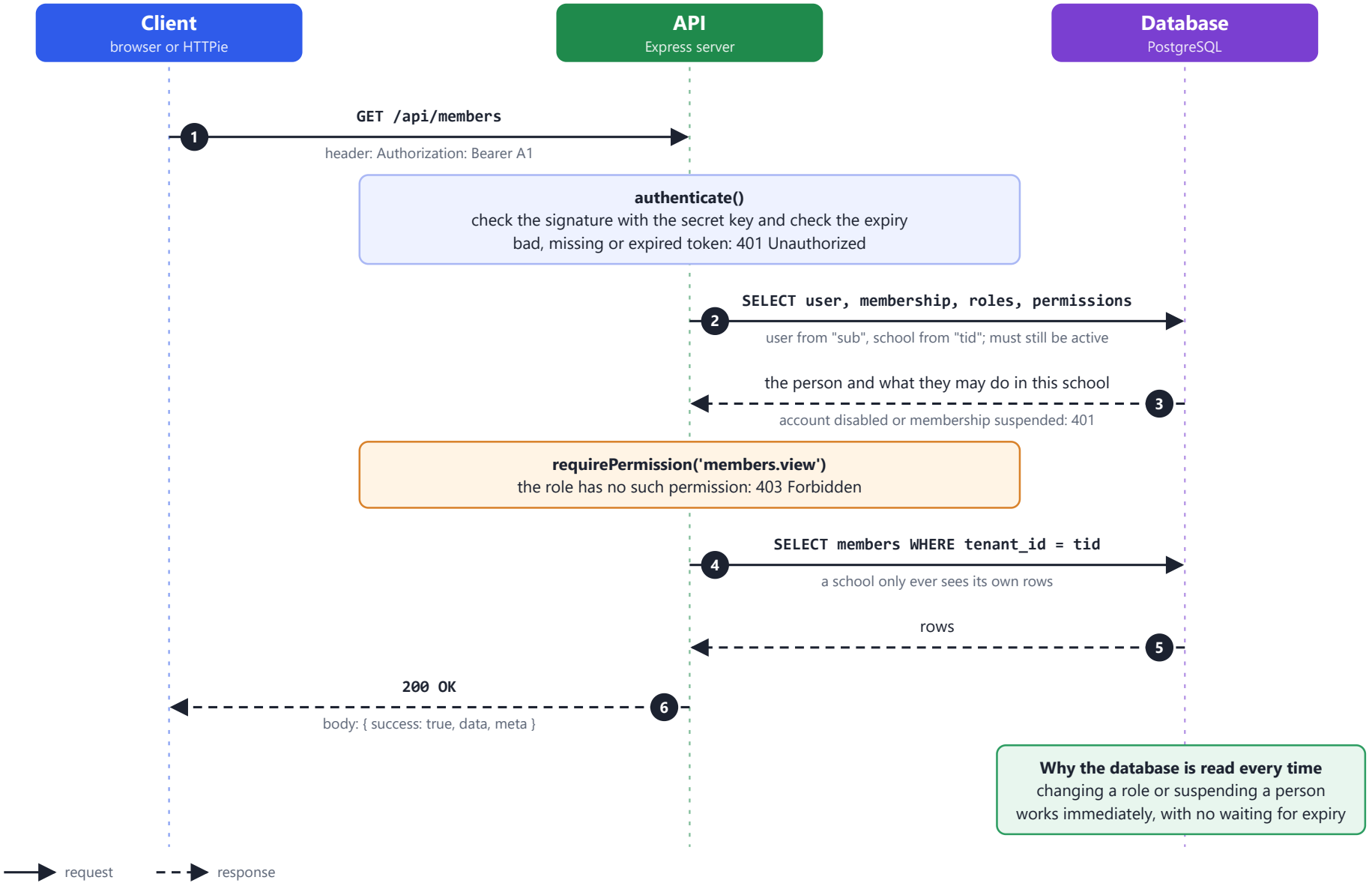
How they work together over time



Every refresh gives a new access token **and** a new cookie with a fresh 30 days, so an active person stays signed in; a login that is never used expires after 30 days.

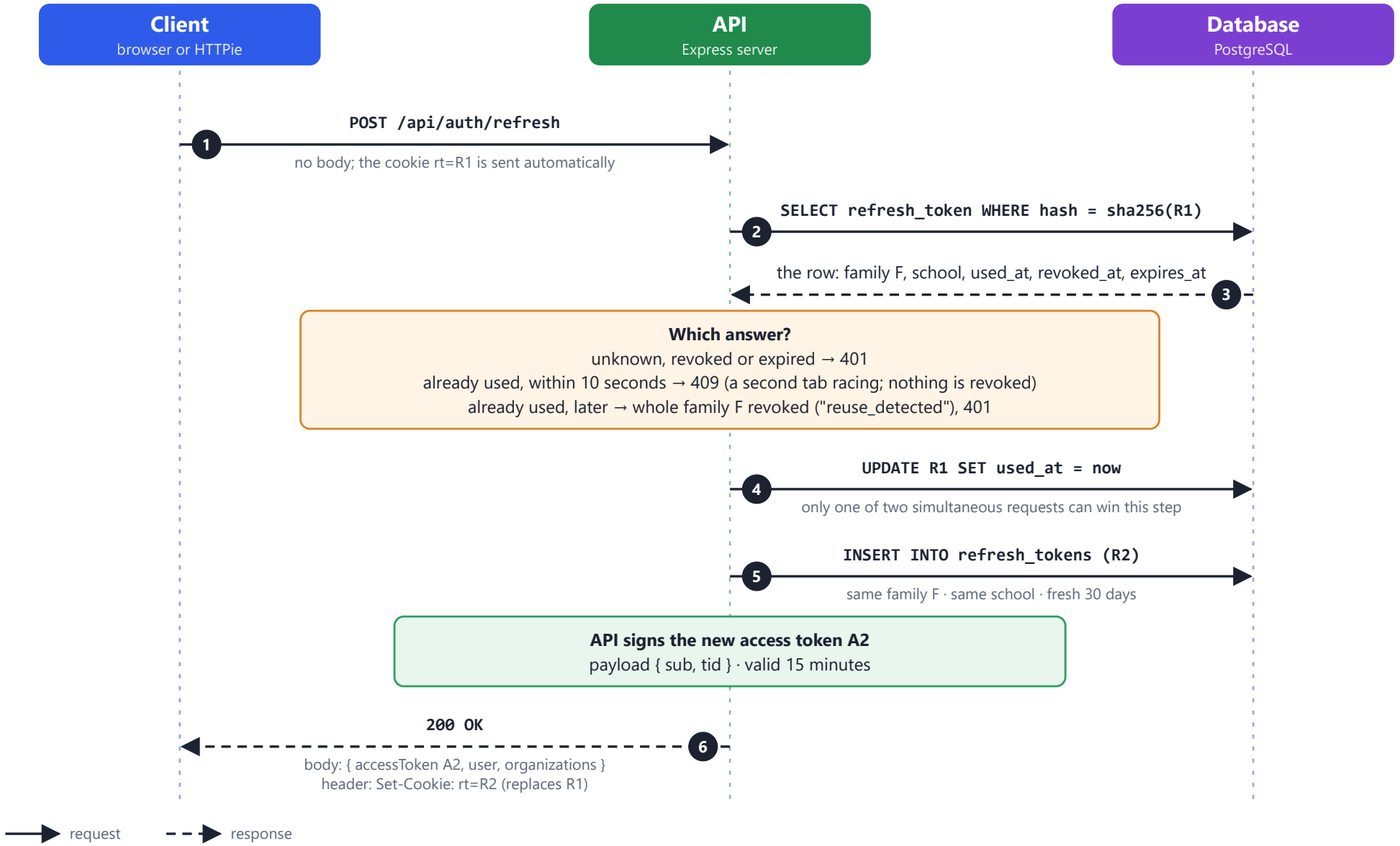
4. Every normal request

The access token proves who you are. The database decides what you may do, every time.



5. Refresh: getting a new access token

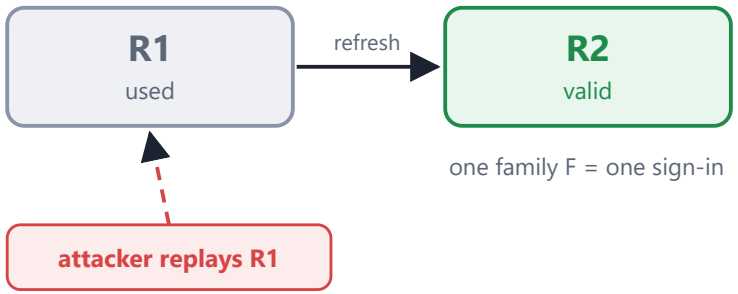
Used when the 15-minute access token has expired. No password is needed: the cookie is the proof.



6. Detecting a stolen cookie

A refresh token works exactly once. If a spent one shows up again, two parties hold the same cookie.

A The old cookie is replayed within 10 seconds



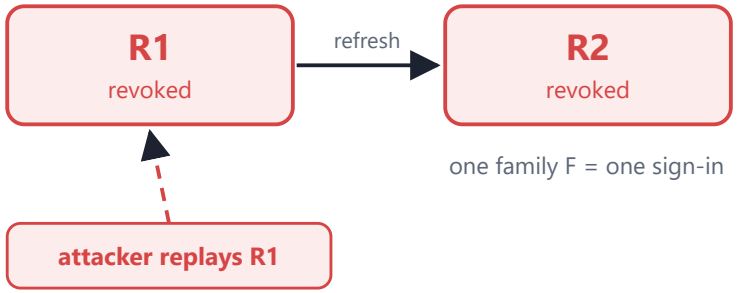
Answer: 409 Conflict

looks like two browser tabs racing

nothing is revoked

the real user simply refreshes again

B The old cookie is replayed later



Answer: 401 Unauthorized

a used token came back: somebody copied it

the whole family F is revoked ("reuse_detected")

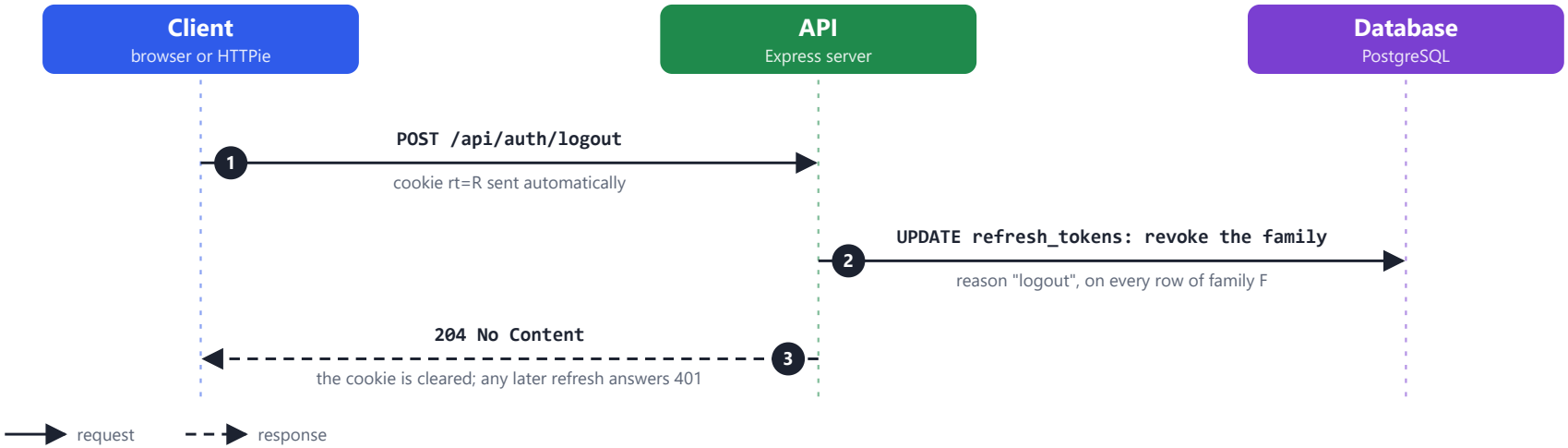
thief and real user are both logged out; the person signs in again

The server cannot tell who is the thief, so it ends the whole login. This is why every refresh swaps the cookie for a new one and the old row is kept in `refresh_tokens` with `used_at` set.

7. Logging out and switching school

Logout ends the server-side session. Switching school issues a new access token for another school.

Log out



Switch school

